

SC102 L04: 协程与数据结构——详细学习笔记

2026 年 5 月 25 日

目录

1	课程概述 [p.1–3]	2
2	枚举器接口: IEnumerator [p.4–5]	2
2.1	什么是枚举器?	2
2.2	IEnumerator 的三个成员	2
2.3	泛型版本 IEnumerator<T>	2
2.4	使用步骤	3
3	可被枚举接口: IEnumerable [p.6]	3
3.1	IEnumerable 的含义	3
3.2	枚举器的实际使用示例 [p.7]	3
4	迭代器: Iterator 与 yield [p.8–10]	4
4.1	什么是迭代器?	4
4.2	yield return 的奇妙之处 [p.8]	4
4.3	yield 的底层原理 [p.9]	4
4.4	迭代器详细示例 [p.10]	5
5	协程的原理 [p.11]	6
5.1	从迭代器到协程	6
5.2	StartCoroutine 的内部原理	7
5.3	两个小问题的解答 [p.11]	7
6	数据结构总览 [p.12, 16, 21]	7
7	多维数组 Multidimensional Array [p.13–14]	7
7.1	维度的含义 [p.13]	7

目录	2
7.2 C# 中的多维数组 [p.14]	8
7.3 访问与遍历 [p.14]	8
8 交错数组 Jagged Array [p.15]	9
8.1 概念与定义	9
8.2 初始化与访问 [p.15]	9
8.3 多维数组 vs 交错数组	10
9 树 Tree [p.17]	10
9.1 概念	10
9.2 C# 实现 [p.17]	10
9.3 树的遍历 [p.17]	10
10 栈 Stack [p.18]	11
10.1 概念: LIFO	11
10.2 三个基本操作 [p.18]	11
10.3 C# 中的 Stack<T> [p.18]	11
11 队列 Queue [p.19]	12
11.1 概念: FIFO	12
11.2 两个基本操作 [p.19]	12
11.3 C# 中的 Queue<T> [p.19]	12
12 选择正确的数据结构 [p.20]	12
13 图算法: DFS 与 BFS [p.22–25]	13
13.1 问题引入: 五子棋胜负判定 [p.22]	13
13.2 深度优先搜索 DFS [p.23]	13
13.3 广度优先搜索 BFS [p.24]	14
13.4 DFS vs BFS 对比 [p.25]	15
14 课后习题详细解答 [p.26]	15
14.1 习题 1: 五子棋胜负判定	15
14.2 习题 2: 实现自定义泛型队列	17
14.3 习题 3: 实现自定义 StartCoroutine	20

1 课程概述 [p.1-3]

本课是 SC-102 Unity 游戏开发 I 的第 4 课，隶属于第一单元“技术基础”，主题为协程（Coroutine）与数据结构（Data Structures）。课程分为两大板块：

- (1) 协程模块：从枚举器（IEnumerator）和迭代器（Iterator）讲起，逐步揭示 Unity 协程的底层实现原理。
- (2) 数据结构模块：涵盖多维数组（Multidimensional Array）、交错数组（Jagged Array）、树（Tree）、栈（Stack）、队列（Queue）以及图算法（Graph Algorithms）中的深度优先搜索（DFS）与广度优先搜索（BFS）。

2 枚举器接口：IEnumerator [p.4-5]

2.1 什么是枚举器？

枚举器接口（IEnumerator Interface）是 .NET 中提供了一种通用遍历集合的方法。当你使用 `foreach` 循环遍历一个集合时，编译器在幕后正是使用了 `IEnumerator` 来完成遍历工作。可以把枚举器想象成一个指向集合中某个元素的“指针”——它知道当前指向哪个元素，并且可以移动到一个元素。

2.2 IEnumerator 的三个成员

`IEnumerator` 接口由三个核心成员组成 [p.5]：

```
1 public interface IEnumerator
2 {
3     bool MoveNext();           // 前进至下一个元素
4     object Current { get; }    // 获取当前元素
5     void Reset();             // 重置回第一个元素
6 }
```

MoveNext() 将枚举器前进到集合中的下一个元素。如果成功前进到下一个元素，返回 `true`；如果已经遍历到集合末尾（没有更多元素），返回 `false`。这是遍历循环的“发动机”。

Current 一个只读属性，返回枚举器当前所在位置的元素。注意：在首次调用 `MoveNext()` 之前访问 `Current` 是未定义行为，通常会抛出异常。

Reset() 将枚举器重置到初始位置（第一个元素之前）。这个方法在 .NET 中很少被实际使用，甚至许多实现会直接抛出 `NotSupportedException`。

2.3 泛型版本 IEnumerator<T>

为了避免装箱（Boxing）带来的性能损耗，.NET 还提供了泛型版本的枚举器接口 [p.5]：

```
1 public interface IEnumerator<T> : IEnumerator
2 {
3     new T Current { get; } // 类型安全的 Current, 隐藏父类的 object Current
4 }
```

使用泛型版本时，Current 直接返回 T 类型，无需从 object 进行强制类型转换。

2.4 使用步骤

使用枚举器的典型步骤为 [p.5]：

- (1) 拿到一个集合（Array、List、Dictionary 等）。
- (2) 获取该集合的枚举器（通过 GetEnumerator() 方法）。
- (3) 使用 Current 访问当前元素，用 MoveNext() 移至下一个元素，直到遍历完成。

3 可被枚举接口：IEnumerable [p.6]

3.1 IEnumerable 的含义

IEnumerable 接口代表一个对象“可被枚举”（Enumerable）。它的定义极其简洁 [p.6]：

```
1 public interface IEnumerable
2 {
3     IEnumerator GetEnumerator();
4 }
```

实现此接口的类必须提供一个返回 IEnumerator 的 GetEnumerator() 方法。C# 中几乎所有内置集合类型都实现了 IEnumerable，包括 Array、List<T>、Dictionary<K,V>、Stack<T>、Queue<T> 等。

IEnumerable 意味着三件事 [p.6]：

- (1) 我们可以获取一个它的枚举器（IEnumerator）。
- (2) 我们可以通过该枚举器遍历其中的元素。
- (3) 我们可以使用 foreach 循环遍历其中的元素——这实际上是前两步的语法糖。

3.2 枚举器的实际使用示例 [p.7]

以下是使用枚举器手动遍历一个 List<int> 的完整示例：

```
1 IEnumerable<int> sequence = new List<int> { 1, 2, 3, 4, 5 };
2 // List<int> 实现了 IEnumerable<int>, 所以可以赋值给该接口类型
3
```

```
4 IEnumerator<int> enumerator = sequence.GetEnumerator();
5 // 调用 GetEnumerator() 获取枚举器
6
7 while (enumerator.MoveNext()) // 每次调用 MoveNext() 前进一位
8 {
9     int current = enumerator.Current; // Current 获取当前位置的元素
10    Console.WriteLine(current);       // 输出: 1 2 3 4 5
11 }
```

理解要点：枚举器就像一个“指针”，初始位置在第一个元素之前。每次调用 `MoveNext()` 都将指针向前移动一位，`Current` 则总是返回指针当前指向的元素。当 `MoveNext()` 返回 `false` 时，说明已经遍历完毕。

4 迭代器：Iterator 与 yield [p.8-10]

4.1 什么是迭代器？

迭代器（Iterator）不是一个接口，也不是一个类，而是一种设计模式（或语言特性）。它允许开发者自定义一个枚举器的行为，而不需要手动实现 `IEnumerator` 或 `IEnumerable` 接口的所有细节。

在 C# 中，迭代器通过 `yield return` 和 `yield break` 关键字来实现。

4.2 yield return 的奇妙之处 [p.8]

观察以下代码，你会发现几个“不对劲”的地方：

```
1 public IEnumerable<int> GetNumbers()
2 {
3     for (int i = 1; i <= 5; i++)
4     {
5         yield return i; // 这里使用了 yield return
6     }
7     // 注意：没有显式的 return 语句！
8 }
```

两个反常之处 [p.8]：

- (1) 函数声明返回 `IEnumerable<int>`，但函数体内没有 `return` 语句。
- (2) 我们没有创建任何实现了 `IEnumerable<int>` 的对象。

那么到底发生了什么？答案是：编译器为我们做了大量幕后工作。

4.3 yield 的底层原理 [p.9]

当编译器在一个返回 `IEnumerable` 或 `IEnumerator` 的方法中遇到 `yield return` 关键字时，会自动生成一个实现了 `IEnumerable/IEnumerator` 的匿名类。这个由编译器生成的类完成两件核

心工作：

- (1) **状态记录 (State Machine)**: 编译器将方法体转换为一个状态机 (State Machine)，它会记录当前的迭代位置和所有局部变量的值。每个 `yield return` 对应状态机的一个状态。当方法被再次调用 (即 `MoveNext()` 被再次调用)，执行会从上上次 `yield return` 之后的位置继续，而不是从头开始。
- (2) **值传递**: 每个 `yield return x` 被翻译为：“将 `Current` 设置为 `x`，记录当前状态，然后暂停执行并返回 `true`。”

核心理解: `yield return` 做两件事——(1) 将枚举器的 `Current` 设为一个值 (提供下一个要枚举的元素)；(2) “暂停”函数的执行，并在下一次调用 `MoveNext()` 时从 `yield return` 的下一行继续执行。

4.4 迭代器详细示例 [p.10]

下面的例子完整展示了 `yield return` 导致的“交替执行”效果：

```
1 var numbers = ProduceEvenNumbers(5);
2 Console.WriteLine("Caller: about to iterate.");
3 foreach (int i in numbers)
4 {
5     Console.WriteLine($"Caller: {i}");
6 }
7
8 IEnumerable<int> ProduceEvenNumbers(int upto)
9 {
10     Console.WriteLine("Iterator: start.");
11     for (int i = 0; i <= upto; i += 2)
12     {
13         Console.WriteLine($"Iterator: about to yield {i}");
14         yield return i;           // 暂停，交出 i
15         Console.WriteLine($"Iterator: yielded {i}"); // 下次从此继续
16     }
17     Console.WriteLine("Iterator: end.");
18 }
```

程序输出：

```
Caller: about to iterate.
Iterator: start.
Iterator: about to yield 0
Caller: 0
Iterator: yielded 0
Iterator: about to yield 2
```

```
Caller: 2
Iterator: yielded 2
Iterator: about to yield 4
Caller: 4
Iterator: yielded 4
Iterator: end.
```

执行顺序分析：

- (1) 调用方（Caller）开始遍历。
- (2) `foreach` 内部首次调用 `MoveNext()`，触发迭代器方法的执行。
- (3) 迭代器输出 “Iterator: start.”，进入循环。
- (4) 遇到 `yield return 0`，`Current` 设为 0，控制权返回调用方。
- (5) 调用方输出 “Caller: 0”。
- (6) 下一次 `MoveNext()`，迭代器从 `yield return 0` 之后继续执行，输出 “Iterator: yielded 0”。
- (7) 循环继续，`i` 变为 2，遇到 `yield return 2`，再次暂停。
- (8)如此交替执行，直到循环结束。

这个例子清晰地展示了迭代器的“暂停-恢复”机制：调用方和迭代器的方法体在交替执行。

5 协程的原理 [p.11]

5.1 从迭代器到协程

Unity 中的协程（Coroutine）巧妙地利用了 C# 迭代器的机制，但它不是用来“遍历集合”，而是用来“执行一系列跨帧的操作”。

关键洞察 [p.11]： 在一个协程函数中，所有的 `yield return` 返回值构成了一个虚拟的“集合”。例如，以下协程：

```
1 IEnumerator MyCoroutine()
2 {
3     // 真正有用的语句 A
4     yield return null;
5     // 真正有用的语句 B
6     yield return new WaitForSeconds(5);
7 }
```

它的“集合”就是 `{null, new WaitForSeconds(5)}`——两个 `yield return` 的返回值。当我们在遍历这个集合时（调用 `MoveNext()`），所有真正有用的语句（A 和 B）都会在两次遍历之间被一并执行。

5.2 StartCoroutine 的内部原理

当我们在 Unity 中调用 `StartCoroutine(MyCoroutine())` 时，Unity 引擎实际上做了两件事 [p.11]:

- (1) **每帧调用 `MoveNext()`**: 每一帧 (Update 之后)，Unity 调用协程返回的 `IEnumerator` 的 `MoveNext()` 方法。这会执行协程中的代码，直到遇到下一个 `yield return`。
- (2) **解释 `Current` 值**: Unity 检查 `Current` 属性 (即 `yield return` 的返回值)，根据不同类型的值做出不同处理:
 - `null`: 表示“本帧已经执行完毕”，下一次 `MoveNext()` 将在下一帧调用。这就是 `yield return null` 的含义——延迟到下一帧。
 - `WaitForSeconds(n)`: 表示“等待 n 秒后再继续”。Unity 记录下等待结束的时间，在时间到达之前不会调用 `MoveNext()`。
 - `WaitForEndOfFrame`: 等到本帧渲染结束后再继续。
 - `WaitUntil(predicate)`: 等到某个条件为真时再继续。

5.3 两个小问题的解答 [p.11]

问题 (1): 为什么 `yield return null` 的作用是延迟到下一帧执行协程?

答: 因为 Unity 每帧调用一次 `MoveNext()`。当 `yield return null` 执行后，`MoveNext()` 返回 `true` (表示还有更多元素)，Unity 检查 `Current` 值为 `null`。对于 `null`，Unity 的约定是“不做特殊等待”，所以下一次 `MoveNext()` 会在下一帧被调用。这就实现了“延迟一帧”的效果。

问题 (2): 可以把 `yield return null` 替换成 `yield return 1` 或 `yield return "Hello"` 吗?

答: 可以。从 C# 语法上讲，协程返回的是 `IEnumerator` (非泛型)，`Current` 类型是 `object`，所以 `yield return` 后可以跟任意类型。但是，Unity 不认识 `1` 或 `"Hello"`，它们会被当作“无特殊含义”的值处理，效果等同于传入了一个自定义对象——Unity 会忽略它，行为上类似 `null` (即等到下一帧继续)。不过不建议这样做，因为这会降低代码的可读性。

6 数据结构总览 [p.12, 16, 21]

进入课程的第二个板块：数据结构。本节课涵盖了多维数组、树、栈、队列，以及图算法中的 DFS 和 BFS。

7 多维数组 Multidimensional Array [p.13–14]

7.1 维度的含义 [p.13]

在一维数组中，我们使用 **1** 个索引来定位元素：


```

1 int[] arr = new int[7];
2 arr[0]    // 使用 1 个 index 索引

```

在 n 维数组中，我们使用 n 个索引来定位元素。维度的直观理解是：索引一个元素需要提供的坐标数量。例如：

- 一维数组是“一条线”：需要 x 坐标。
- 二维数组是“一个面”（如棋盘）：需要 (x, y) 坐标。
- 三维数组是“一个体”（如体素/方块）：需要 (x, y, z) 坐标。

7.2 C# 中的多维数组 [p.14]

在 C# 中，使用逗号分隔的方括号语法定义多维数组：

```

1 int[,] grid2D = new int[5, 5];           // 5x5 的二维数组
2 int[,,] grid3D = new int[7, 8, 9];      // 7x8x9 的三维数组
3 int[,,,] grid4D = new int[1, 2, 3, 4];  // 1x2x3x4 的四维数组

```

重要：定义多维数组时必须显式指定每一维的大小，因为 C# 在堆上为多维数组分配连续内存块，编译器需要知道总元素数量。

7.3 访问与遍历 [p.14]

访问元素：每个维度的索引从 0 开始，访问时提供所有维度的索引：

```

1 int[,] grid = new int[4, 2]; // 4行 x 2列，共8个元素
2 grid[0, 0] // 第0行第0列 —— 合法
3 grid[3, 1] // 第3行第1列 —— 合法
4 grid[4, 1] // 越界！行索引最大为 3
5 grid[0, 2] // 越界！列索引最大为 1

```

遍历多维数组：使用嵌套循环，通过 `GetLength(dimension)` 获取指定维度的长度：

```

1 int[,] grid = new int[4, 5];
2 for (int i = 0; i < grid.GetLength(0); i++) // GetLength(0) = 4
3 {
4     for (int j = 0; j < grid.GetLength(1); j++) // GetLength(1) = 5
5     {
6         grid[i, j] = i * 10 + j; // 赋值
7         Console.Write(grid[i, j] + " ");
8     }
9     Console.WriteLine();
10 }

```

8 交错数组 Jagged Array [p.15]

8.1 概念与定义

交错数组（Jagged Array）是“数组的数组”（Array of Arrays）。与多维数组不同，交错数组的每一个子数组可以拥有不同的长度，形成“锯齿形”的结构。用 `[] []` 语法定义：

```
1 int[] [] jagged = new int[5] []; // 5 个子数组，目前每个都是 null
```

注意区别：

- `int[,]` 是“一个”多维数组对象，分配连续内存。所有行长度相同。
- `int[] []` 是“一个”外数组 + “多个”独立的内数组对象。每行可以不同长度。

8.2 初始化与访问 [p.15]

由于子数组默认是 `null`，需要手动初始化每个子数组：

```
1 int[] [] jagged = new int[3] [];  
2 jagged[0] = new int[4]; // 第0行，4个元素  
3 jagged[1] = new int[2]; // 第1行，2个元素  
4 jagged[2] = new int[5]; // 第2行，5个元素  
5  
6 // 访问元素  
7 jagged[0][2] // 第0行第2个元素 —— 合法  
8 jagged[1][3] // 越界！第1行只有2个元素
```

遍历交错数组：

```
1 for (int i = 0; i < jagged.Length; i++) // jagged.Length = 3  
2 {  
3     for (int j = 0; j < jagged[i].Length; j++) // jagged[i].Length 因行而异  
4     {  
5         jagged[i][j] = 0;  
6     }  
7 }
```

注意遍历交错数组时，内层循环的上限是 `jagged[i].Length`（每一行的独立长度），而不是统一的 `GetLength(1)`。

8.3 多维数组 vs 交错数组

特性	多维数组 <code>int[,]</code>	交错数组 <code>int[][]</code>
内存布局	连续内存块	分散的多个数组
行长度	必须全部相同	可以不同
初始化	声明时一次分配	需逐行初始化
访问语法	<code>arr[i,j]</code>	<code>arr[i][j]</code>
性能	更好的缓存局部性	可能更快 (JIT 优化更好)

9 树 Tree [p.17]

9.1 概念

树 (Tree) 是一种层级化的数据结构，由节点 (Node) 组成。树有一个根节点 (Root Node)，每个节点可以有零个或多个子节点 (Child Node)。游戏中常见的“技能树”、“科技树”、“场景层级”都是树的实例。

树实际上是一种特殊的有向图 (Directed Graph)，它允许我们表示物体之间的依赖关系和层级关系。

9.2 C# 实现 [p.17]

每个树节点可以用如下数据结构表示：

```

1 class TreeNode
2 {
3     public float value;
4     public List<TreeNode> children = new List<TreeNode>();
5 }

```

`value` 存储节点自身的数据，`children` 是一个 `List<TreeNode>`，存储所有子节点的引用。

9.3 树的遍历 [p.17]

从根节点出发，通过递归可以遍历整棵树：

```

1 void Traversal(TreeNode root)
2 {
3     if (root == null) return;
4     Console.WriteLine(root.value);           // 访问当前节点
5     foreach (var child in root.children)
6     {
7         Traversal(child);                     // 递归访问每个子节点
8     }
9 }

```

这种“先访问根节点,再递归访问所有子节点”的方式称为**深度优先遍历**(Depth-First Traversal)——这正是我们在后面要讲的 DFS 算法在树结构上的具体应用。

一个值得注意的联系是: C# 的 GC (垃圾回收) 的标记算法就使用了树/图的遍历来标记所有存活的对象。

10 栈 Stack [p.18]

10.1 概念: LIFO

栈(Stack)是一种“先进后出”(LIFO: Last In, First Out)的线性数据结构。可以把它想象成一个**盒子**: 数据依次堆叠在盒子里,你只能从最上方放入或取出物品,不能从中间操作。

这里所说的栈与内存中的**调用栈**(Call Stack)本质上是同样的数据结构——函数调用时,返回地址和局部变量被“压入”调用栈;函数返回时,它们被“弹出”。

10.2 三个基本操作 [p.18]

Push(object obj) 将一个元素压入栈顶。

Pop() 将栈顶元素移除并返回。如果栈为空则会抛出异常 (InvalidOperationException)。

Peek() 返回栈顶元素但不移除它。用于“看一眼”最顶上的元素。

10.3 C# 中的 Stack<T> [p.18]

C# 提供了泛型的栈类 Stack<T> (位于 System.Collections.Generic 命名空间):

```
1 using System.Collections.Generic;
2
3 Stack<int> stack = new Stack<int>();
4
5 stack.Push(10);    // 栈: [10]
6 stack.Push(20);    // 栈: [10, 20]
7 stack.Push(30);    // 栈: [10, 20, 30]
8
9 int top = stack.Peek(); // top = 30, 栈不变
10 int popped = stack.Pop(); // popped = 30, 栈: [10, 20]
11 Console.WriteLine(stack.Count); // 2
```

应用场景:

- 撤销操作 (Undo): 每次操作 Push 到栈,撤销时 Pop。
- 函数调用栈: 编译器/运行时管理函数调用和返回。
- 括号/标签匹配: 遇到开括号 Push, 遇到闭括号时与栈顶 Pop 出的开括号匹配。

11 队列 Queue [p.19]

11.1 概念：FIFO

队列（Queue）是一种“先进先出”（FIFO: First In, First Out）的线性数据结构，与栈相反。可以把它想象成一条**隧道**：数据依次排在隧道中，只有最前面（队头）的元素可以被取出，新元素只能加在最后面（队尾）。

11.2 两个基本操作 [p.19]

Enqueue(object obj) 将一个元素放入队尾（入队）。

Dequeue() 将队头的元素移除并返回（出队）。如果队列为空则抛出异常。

11.3 C# 中的 Queue<T> [p.19]

C# 提供了泛型的队列类 Queue<T>:

```
1 using System.Collections.Generic;
2
3 Queue<int> queue = new Queue<int>();
4
5 queue.Enqueue(10);    // 队列: [10]
6 queue.Enqueue(20);    // 队列: [10, 20]
7 queue.Enqueue(30);    // 队列: [10, 20, 30]
8
9 int front = queue.Dequeue(); // front = 10, 队列: [20, 30]
10 Console.WriteLine(queue.Count); // 2
```

应用场景:

- 消息队列 / 任务队列：按到达顺序依次处理。
- BFS 遍历：广度优先搜索使用队列来管理层次遍历。
- 输入缓冲：按键输入按顺序处理。

12 选择正确的数据结构 [p.20]

课件给出了三个实际场景，需要选择合适的结构：

场景 1：叠放的 UI 界面。玩家只能操作最上方的界面（打开新界面时它盖在上面，关闭时回到下一层）。这对应**后进先出**的模式，应使用**栈（Stack）**。打开界面时 Push，关闭界面时 Pop。

场景 2：新物品提示队列。获得新物品时，提示按顺序逐个显示（前一个消失后才显示后一个）。这对应**先进先出**的模式，应使用**队列（Queue）**。新物品信息 Enqueue，每次取一个 Dequeue 显示，显示完后取下一个。

场景 3：飞船科技树。科技有分支，每个分支中的升级需要依次解锁。这是一种层级依赖关系，应使用树（Tree）。根节点是初始科技，子节点是需要前置科技解锁的升级。

13 图算法：DFS 与 BFS [p.22–25]

13.1 问题引入：五子棋胜负判定 [p.22]

考虑一个 $N \times N$ 的五子棋棋盘，用二维数组表示：

- 0 = 空位
- 1 = 白棋
- 2 = 黑棋

问题：如何判断棋盘上是否有某一方获胜（五子连成一线）？

一种思路是：从每一个非零位置开始，向 8 个方向（上、下、左、右、四个对角线）“发出一条射线”，如果某个方向上连续遇到 5 个相同颜色的棋子，则判断该方获胜。这本质上就是图搜索的思想——将棋盘上的每个位置视为一个节点，相邻位置之间有一条边。

13.2 深度优先搜索 DFS [p.23]

核心思想：选一个方向，一路走到底；遇到死路了，回到上一个路口换方向继续。就像走迷宫时沿着一面墙一直走。

前面讲述的树遍历算法本质上就是 DFS：

```
1 void Traversal(TreeNode root)
2 {
3     if (root == null) return;
4     Console.WriteLine(root.value);
5     foreach (var child in root.children)
6         Traversal(child); // 递归深入每一支
7 }
```

通用的 DFS 模板（带访问标记）[p.23]：

```
1 void DepthFirstSearch(TreeNode root)
2 {
3     if (root == null) return;
4     root.visited = true; // 标记当前节点为已访问
5     foreach (var child in root.children)
6     {
7         if (!child.visited) // 只访问未访问过的节点
8         {
9             DepthFirstSearch(child);
10        }
```

```
11     }  
12 }
```

为什么需要 **visited** 标记？如果没有标记，遍历可能陷入无限循环（在有环的图中）。即使是无环的树，多个父节点指向同一子节点时，没有标记会导致重复访问和指数级的时间开销。

13.3 广度优先搜索 BFS [p.24]

核心思想：不急于深入，而是“一层一层”地向外扩展。就像在水面扔一颗石子，波纹逐层扩散。

BFS 使用队列（**Queue**）来管理“当前层的节点”，这是其与 DFS 在实现上的最大区别。

通用的 BFS 模板 [p.24]：

```
1 void BreadthFirstSearch(Node root)  
2 {  
3     Queue<Node> queue = new Queue<Node>();  
4     root.visited = true;  
5     queue.Enqueue(root);           // 将起点入队  
6  
7     while (queue.Count > 0)        // 只要队列不为空  
8     {  
9         Node node = queue.Dequeue(); // 取出队头节点  
10        // 处理 node...  
11  
12        foreach (var child in node.children)  
13        {  
14            if (!child.visited)  
15            {  
16                child.visited = true; // 标记为已访问  
17                queue.Enqueue(child); // 子节点入队  
18            }  
19        }  
20    }  
21 }
```

为什么 **BFS** 使用队列？队列的 FIFO 特性确保了“先被发现的节点先被处理”，这恰好对应了“逐层扩展”的需求：第一层的所有节点先入队，它们出队时第二层的节点入队，然后是第三层……从而实现了层序遍历。

13.4 DFS vs BFS 对比 [p.25]

特性	DFS	BFS
实现方式	递归 / 栈 (Stack)	队列 (Queue)
遍历顺序	纵向深入	横向扩展
空间复杂度	O(深度)	O(宽度)
找最短路径	不一定	可以 (无权图中)
检测连通性	可以	可以

场景分析 [p.25]:

- (1) **范围攻击**: 在地图的 5×5 方格内找到所有被攻击到的敌人。这是一个“从中心向四周扩展”的问题，应使用 **BFS**（或直接按曼哈顿距离遍历）。BFS 天然适合“逐层扩散”。
- (2) **查找子物体**: 判断物体的所有后代中是否有名为“Box”的物体。这是逐层还是深入均可，但 **DFS** 更适合树形层级结构（Unity 的 Transform 层级本质就是树）。递归 DFS 实现也更简洁。
- (3) **迷宫路径**: 找到从起点到终点的任意路径，DFS 和 BFS 都可以。但要找最短路径，在无权图中必须用 **BFS**（因为 BFS 首次发现终点时走过的步数就是最短路径长度）。

14 课后习题详细解答 [p.26]

14.1 习题 1: 五子棋胜负判定

题目: 使用 DFS 或 BFS 实现判断五子棋是否有玩家获胜的算法。

解题思路:

五子棋的获胜条件是任意方向（水平、垂直、两条对角线）上连续 5 个相同颜色的棋子。最简洁的做法是：对棋盘上的每一个非空位置，检查从它出发的 4 个方向（右、下、右下、左下）是否连续有 5 个相同棋子。注意只需要检查 4 个方向而不是 8 个，因为其他 4 个方向已经被对称覆盖了（例如向左找等价于从更左边的位置向右找）。

```

1 public static bool CheckWin(int[,] board)
2 {
3     int rows = board.GetLength(0);
4     int cols = board.GetLength(1);
5
6     // 四个方向: (行偏移, 列偏移)
7     // 右、下、右下对角线、左下对角线
8     int[,] directions = { {0,1}, {1,0}, {1,1}, {1,-1} };
9
10    for (int r = 0; r < rows; r++)
11    {
12        for (int c = 0; c < cols; c++)
13        {
14            int current = board[r, c];

```



```

15         if (current == 0) continue; // 空位跳过
16
17         // 对每个方向检查
18         for (int d = 0; d < directions.GetLength(0); d++)
19         {
20             int dr = directions[d, 0];
21             int dc = directions[d, 1];
22             int count = 1; // 当前棋子自身算1个
23
24             // 向正方向延伸
25             for (int step = 1; step < 5; step++)
26             {
27                 int nr = r + dr * step;
28                 int nc = c + dc * step;
29                 if (nr >= 0 && nr < rows && nc >= 0 && nc < cols
30                     && board[nr, nc] == current)
31                 {
32                     count++;
33                 }
34                 else
35                 {
36                     break;
37                 }
38             }
39
40             if (count >= 5)
41             {
42                 Console.WriteLine($"Player {current} wins!");
43                 return true;
44             }
45         }
46     }
47 }
48 return false;
49 }

```

复杂度分析：时间复杂度 $O(N^2 \times 4 \times 5) = O(N^2)$ ，对于 $N = 15$ 的标准棋盘完全可接受。

使用 DFS 的替代实现：

如果一定要用 DFS 来实现，可以从每个棋子出发，沿着一个方向递归搜索连续相同棋子的数量：

```

1 public static bool CheckWinDFS(int[,] board)
2 {
3     int rows = board.GetLength(0);
4     int cols = board.GetLength(1);
5     int[] dr = { 0, 1, 1, 1 };

```

```

6      int[] dc = { 1, 0, 1, -1 };
7
8      for (int r = 0; r < rows; r++)
9      {
10         for (int c = 0; c < cols; c++)
11         {
12             if (board[r, c] == 0) continue;
13
14             for (int d = 0; d < 4; d++)
15             {
16                 if (CountConsecutive(board, r, c, dr[d], dc[d],
17                     board[r, c]) >= 5)
18                 {
19                     return true;
20                 }
21             }
22         }
23     }
24     return false;
25 }
26
27 static int CountConsecutive(int[,] board, int r, int c,
28     int dr, int dc, int player)
29 {
30     // 递归基：越界或不是同一玩家的棋子
31     if (r < 0 || r >= board.GetLength(0) ||
32         c < 0 || c >= board.GetLength(1) ||
33         board[r, c] != player)
34     {
35         return 0;
36     }
37     // 递归：往该方向继续搜索
38     return 1 + CountConsecutive(board, r + dr, c + dc, dr, dc, player);
39 }

```

14.2 习题 2：实现自定义泛型队列

题目：不使用 C# 的 `Queue<T>` 类，编写自己的泛型队列类，支持 `Enqueue()` 和 `Dequeue()` 方法。

解题思路：

自定义队列可以使用循环数组（Circular Array）或链表（Linked List）来实现。下面是基于链表的实现（更简洁直观）和基于循环数组的实现（更高效）。

方案 A：基于链表（LinkedList）

```

1 public class MyQueue<T>

```

```
2 {
3     private class Node
4     {
5         public T Value;
6         public Node Next;
7         public Node(T value) { Value = value; }
8     }
9
10    private Node head; // 队头 (最早进入的元素)
11    private Node tail; // 队尾 (最新进入的元素)
12    private int count;
13
14    public int Count => count;
15    public bool IsEmpty => count == 0;
16
17    // 入队: 在队尾添加元素
18    public void Enqueue(T item)
19    {
20        Node newNode = new Node(item);
21        if (tail == null)
22        {
23            // 队列为空: 头尾都指向新节点
24            head = tail = newNode;
25        }
26        else
27        {
28            tail.Next = newNode; // 当前的尾节点指向新节点
29            tail = newNode;      // 更新尾节点引用
30        }
31        count++;
32    }
33
34    // 出队: 移除并返回队头元素
35    public T Dequeue()
36    {
37        if (IsEmpty)
38            throw new InvalidOperationException("Queue is empty.");
39
40        T value = head.Value;
41        head = head.Next; // 头节点后移
42        count--;
43
44        if (head == null) // 如果队列变空, 尾节点也要置空
45            tail = null;
46
47        return value;
48    }
49 }
```

```
48     }
49
50     // 查看队头元素（不移除）
51     public T Peek()
52     {
53         if (IsEmpty)
54             throw new InvalidOperationException("Queue is empty.");
55         return head.Value;
56     }
57 }
```

方案 B：基于循环数组（Circular Array）

```
1 public class MyQueue<T>
2 {
3     private T[] items;
4     private int head;    // 队头索引
5     private int tail;    // 队尾索引（下一个入队位置）
6     private int count;
7
8     public MyQueue(int capacity = 4)
9     {
10         items = new T[capacity];
11         head = 0;
12         tail = 0;
13         count = 0;
14     }
15
16     public int Count => count;
17
18     public void Enqueue(T item)
19     {
20         if (count == items.Length)
21             Resize(items.Length * 2); // 扩容
22
23         items[tail] = item;
24         tail = (tail + 1) % items.Length; // 循环移动
25         count++;
26     }
27
28     public T Dequeue()
29     {
30         if (count == 0)
31             throw new InvalidOperationException("Queue is empty.");
32
33         T value = items[head];
34         items[head] = default(T); // 释放引用（帮助GC）
```

```

35     head = (head + 1) % items.Length;
36     count--;
37     return value;
38 }
39
40 private void Resize(int newCapacity)
41 {
42     T[] newItems = new T[newCapacity];
43     for (int i = 0; i < count; i++)
44     {
45         newItems[i] = items[(head + i) % items.Length];
46     }
47     items = newItems;
48     head = 0;
49     tail = count;
50 }
51 }

```

性能对比:

- 链表实现: Enqueue $O(1)$, Dequeue $O(1)$, 但有额外的节点对象内存开销。
- 循环数组实现: Enqueue $O(1)$ 均摊, Dequeue $O(1)$, 内存更紧凑, 缓存友好。

14.3 习题 3: 实现自定义 StartCoroutine

题目: 不使用 Unity 的 StartCoroutine() 方法, 编写自己的 StartCoroutine() 方法, 支持 yield return null 和 yield return new WaitForSeconds(n)。

解题思路:

核心原理在课件第 11 页已经讲述:

- (1) 协程就是一个返回 IEnumerator 的方法。
- (2) 每帧调用协程的 MoveNext(), 执行到下一个 yield return。
- (3) 根据 Current 值的类型决定何时再次调用 MoveNext()。

我们需要:

- 一个自定义的 WaitForSeconds 类来模拟 Unity 的等待。
- 一个协程管理器, 维护活跃协程列表, 每帧执行一次。

```

1 using System;
2 using System.Collections;
3 using System.Collections.Generic;
4

```

```
5 // 自定义的等待类：记录等待的结束时间
6 public class WaitForSeconds
7 {
8     public float EndTime { get; }
9     public WaitForSeconds(float seconds)
10    {
11        EndTime = Time.time + seconds; // 假设有全局 Time.time
12    }
13 }
14
15 // 协程管理器
16 public class CoroutineManager
17 {
18     private struct CoroutineEntry
19     {
20         public IEnumerator enumerator;
21         public float? waitUntil; // null = 下一帧执行
22     }
23
24     private List<CoroutineEntry> coroutines = new List<CoroutineEntry>();
25
26     // 启动协程
27     public void StartCoroutine(IEnumerator coroutine)
28     {
29         coroutines.Add(new CoroutineEntry
30         {
31             enumerator = coroutine,
32             waitUntil = null // 首帧立即执行
33         });
34     }
35
36     // 每帧调用一次（应在 Update 中调用）
37     public void Update()
38     {
39         for (int i = coroutines.Count - 1; i >= 0; i--)
40         {
41             var entry = coroutines[i];
42
43             // 检查是否到了可以执行的时间
44             if (entry.waitUntil.HasValue &&
45                 Time.time < entry.waitUntil.Value)
46             {
47                 continue; // 还没到时间，跳过
48             }
49
50             // 执行协程的下一步
```

```
51         bool hasNext = entry.enumerator.MoveNext();
52
53         if (hasNext)
54         {
55             object current = entry.enumerator.Current;
56             if (current is WaitForSeconds ws)
57             {
58                 entry.waitUntil = ws.EndTime;
59             }
60             else
61             {
62                 // yield return null 或其他值：下一帧执行
63                 entry.waitUntil = null;
64             }
65             coroutines[i] = entry;
66         }
67         else
68         {
69             // MoveNext() 返回 false：协程结束
70             coroutines.RemoveAt(i);
71         }
72     }
73 }
74
75
76 // 简单的 Time 辅助类（在实际 Unity 中由引擎提供）
77 public static class Time
78 {
79     public static float time = 0f;
80     public static float deltaTime = 0.016f; // 约 60fps
81 }
82
83 // ===== 使用示例 =====
84 public class Example
85 {
86     private CoroutineManager manager = new CoroutineManager();
87
88     public void Start()
89     {
90         manager.StartCoroutine(MyCoroutine());
91     }
92
93     public void Update(float dt)
94     {
95         Time.time += Time.deltaTime;
96         manager.Update();
97     }
98 }
```

```
97     }
98
99     private IEnumerator MyCoroutine()
100     {
101         Console.WriteLine($"Frame start: {Time.time:F3}");
102         yield return null; // 等待一帧
103
104         Console.WriteLine($"After 1 frame: {Time.time:F3}");
105         yield return new WaitForSeconds(2.0f); // 等待2秒
106
107         Console.WriteLine($"After 2 seconds: {Time.time:F3}");
108     }
109 }
```

执行流程详解:

- (1) `StartCoroutine()` 将协程的 `IEnumerator` 加入管理列表。
- (2) 第一帧 `Update()`: 调用 `MoveNext()`, 执行到第一个 `yield return null`。 `Current` 为 `null`, 设置 `waitUntil = null`, 表示下一帧继续。
- (3) 第二帧 `Update()`: 再次 `MoveNext()`, 执行到 `yield return new WaitForSeconds(2)`。 `Current` 是 `WaitForSeconds` 对象, 设置 `waitUntil = Time.time + 2`。
- (4) 接下来的约 120 帧 (2 秒 / 0.016 秒每帧), `waitUntil` 检查不通过, 跳过此协程。
- (5) 2 秒后, 条件满足, 调用 `MoveNext()`, 协程继续执行直到结束。 `MoveNext()` 返回 `false`, 协程从列表中移除。

这个实现完整复制了 Unity 协程的核心机制, 帮助理解 `StartCoroutine()` 的内部工作原理。

总结

本课从 C# 的枚举器接口 (`IEnumerator`) 讲起, 逐步揭示了迭代器 (Iterator) 和 `yield return` 的底层原理——编译器生成的状态机。在此基础上, 深入解析了 Unity 协程的实现机制: 协程本质上利用了枚举器的“暂停-恢复”特性, 将 `yield return` 的返回值作为跨帧调度的依据。

课程下半部分转入数据结构, 讲解了多维数组、交错数组、树、栈、队列, 以及图算法中的 DFS 和 BFS。这些数据结构在游戏开发中处处可见: 场景层级是树、UI 层级是栈、消息队列是队列、寻路算法用 BFS/DFS。掌握它们不仅是写代码的基础, 更是设计游戏系统的核心能力。